

Design of Cards Collection

Edward John Steere

<2023-05-26 Fri> June 20, 2023

Contents

1	Introduction	1
2	Firestore Schema	1
3	Considerations Regarding the Updating of Plottr	3
4	Update Strategy	4
5	New Firestore Schema	4
5.1	Query for Cards	5
6	Watching for Invalid Version Changes	6
7	Backwards Compatibility	6
8	Production Considerations	6
8.1	Plottr Mobile	7
9	Conclusion	7

1 Introduction

In Plottr Pro, a file's cards can't exceed 1MB in total. In practice, we've received around one complaint per week relating to users who've exceeded this limit. Some of these users are confused and angry because they lost work when the cards weren't saved.

In the past, we've viewed this as somewhat of an anomaly because we found it hard to imagine that many users would run into this problem, but we're receiving an increasing number of complaints.

This document addresses that concern by proposing a new structure for storing cards on Firestore that grows indefinitely. In solving the problem, it's important to consider whether to maintain backwards compatibility and how to accommodate `plottr_mobile`.

2 Firestore Schema

In Firestore, a Plottr file's cards are stored as an object where each of the keys of the object are integer ids and the values are the cards that have that id. When the cards are read from Firestore

to either Plottr Web, Desktop or Mobile, they're transformed into a flat array and used as the value of the `cards` key in the Redux store.

Figure 2 is an example created from an example file in the `plottr_desktop` repo. The object might be stored at the path: `cards/<fileId>`.

```
{
  "2": {
    "id": 2,
    "lineId": 6,
    "beatId": 7,
    "bookId": null,
    "positionWithinLine": 0,
    "positionInBeat": 0,
    "title": "",
    "description": [
      {
        "type": "paragraph",
        "children": [
          {
            "text": ""
          }
        ]
      }
    ],
    "tags": [],
    "characters": [],
    "places": [],
    "templates": [],
    "imageId": null,
    "fromTemplateId": null
  },
  "1": {
    "id": 1,
    "lineId": 5,
    "beatId": 6,
    "bookId": null,
    "positionWithinLine": 0,
    "positionInBeat": 0,
    "title": "Scene 1",
    "description": [
      {
        "type": "paragraph",
        "children": [
          {
            "text": ""
          }
        ]
      }
    ],
    "tags": [],
    "characters": [],
    "places": [],
    "templates": [],
    "imageId": null,
    "fromTemplateId": null
  }
}
```

There are two known problems with the existing schema. The first relates to broken objects and missing metadata. The second relates to the size limitation.

Regarding the first problem; objects that represent arrays are translated to arrays by `plottr_firebase`. During this process, we discard any root-level entity on the object that wasn't part of an array element. For example, objects that represent arrays have a timestamp and an optional `deleted` field. Both those fields have previously been interpreted as array elements, causing files to be unreadable by `plottr_firebase` until patches were added to the reading logic to prevent misinterpretation. Now that metadata fields are ignored by `plottr_firebase`, the client code can't tell whether an array is part of a deleted file.

Regarding the second problem; as discussed above, the size of the object must not exceed 1MB when stored (in whatever format Firebase Storage uses on their server). Firebase prefers that elements be stored in small documents and for there to be many documents rather than a few larger documents.

This Work primarily addresses the second problem and incidentally address the first problem.

3 Considerations Regarding the Updating of Plottr

Before work begins, we must answer the following questions:

1. What happens when a user updates Plottr?
2. What happens when a user attempts to use a new file from an older version?
3. What happens when mobile attempts to boot the file?
4. What happens when the file is already open in one device and the version changes to a breaking version on another device?

In the feature channel on Slack, we reached a consensus that the file should not be backwards-compatible after this change. This is a somewhat unusual case to break backwards compatibility because the structure of the portable Plottr file (AKA a "classic file") will remain the same. The only changes aren't observable by the user.

The breaking migration is necessary to prevent two strange cases of state management. These are:

- Cards disappearing from a file opened in an version as they're moved into the new collection.
- Older versions writing into the old collection where the newer version wont see them.

Assuming a breaking migration is to be inserted by Plottr, these are possible answers to the questions:

1. Any file opened from this date onwards will undergo migration by immediately migrating all cards to the new schema (bulk) or by migrating cards as their opened (incremental).
2. Plottr displays a message informing the user that it's out of date and how to acquire the update.
3. This is still somewhat of an unknown. The logic to prevent mobile from opening new files when it's too old is untested.
4. Plottr doesn't handle this case gracefully. We ought to check whether the file's version ever changes to something that would invalidate the current session and produce the same update screen as we do when booting a file that's too new.

4 Update Strategy

We learned from the character attributes per book feature not to incrementally migrate files. At the time, we believed that the process would support backwards compatibility. Ultimately, we discovered that the process is very complex and the work to make it completely backwards compatibly was infeasible in our time frames. Ultimately we regretted our decision to incrementally migrate character attributes.

Another consideration is how to apply this migration. We can't migrate the database from the usual place (the `pltr` migration module) because `pltr` isn't responsible for how files are stored. Additionally, Plottr checks the `file` key managed by `pltr` when it decides whether the file is too new.

Here are two proposed changes (that both must be made) to resolve this problem:

1. We should add a breaking migration that does nothing to `pltr`.
2. Every time a file is booted, `plottr_firebase` should check for cards in the old collection and migrate them to the new collection regardless of the version of file opened. (We could also listen for new entries and move them immediately, but I don't think this is necessary and we'll incur monetary costs to listen to more collections than is necessary.)

5 New Firebase Schema

New cards will be stored in `flatCards/<fileId>/cards/<syntheticId>`.

For example, the `flatCards` collection might have the following structure:

```
[
  {
    "234345-EEFF22-2233": [
      {
        "_id": "234345-EEFF22-2233",
        "cards": [
          {
            "id": 2,
            "lineId": 6,
            "beatId": 7,
            "bookId": null,
            "positionWithinLine": 0,
            "positionInBeat": 0,
            "title": "",
            "description": [
              {
                "type": "paragraph",
                "children": [
                  {
                    "text": ""
                  }
                ]
              }
            ]
          },
          {
            "tags": [],
            "characters": [],
            "places": [],
            "templates": [],
            "imageId": null,
            "fromTemplateId": null
          }
        ]
      }
    ]
  }
]
```

```

    }
  ]
},
{
  "_id": "234345-EEFF22-2233",
  "cards": [
    {
      "id": 1,
      "lineId": 5,
      "beatId": 6,
      "bookId": null,
      "positionWithinLine": 0,
      "positionInBeat": 0,
      "title": "Scene 1",
      "description": [
        {
          "type": "paragraph",
          "children": [
            {
              "text": ""
            }
          ]
        }
      ],
      "tags": [],
      "characters": [],
      "places": [],
      "templates": [],
      "imageId": null,
      "fromTemplateId": null
    }
  ]
}
]
}
]
}
]

```

The following Firestore rule restricts access to the new collection as it is for all other collections.

```

match /databases/{database}/documents {
  ...
  match /flatCards/{document}/cards {
    allow read, write, update, create: if request.auth != null && get(/databases/{database}/documents/authorisation/{request.auth.uid}/granted/{document}).data.permission == 'owner';
    allow read, write, update: if request.auth != null && get(/databases/{database}/documents/authorisation/{request.auth.uid}/granted/{document}).data.permission == 'collaborator';
    allow read: if request.auth != null && get(/databases/{database}/documents/authorisation/{request.auth.uid}/granted/{document}).data.permission == 'viewer';
  }
  ...
}

```

5.1 Query for Cards

Plottr queries cards according to which file they belong to. We don't need to transform the payload in `plottr_firestore` anymore because it'll already have the correct type: an array.

The following listing queries Firebase for all the cards belonging to a particular file.

```
const listenToFlatArrayAtPath =
  (path) =>
    (userId, fileId, clientId, withAction, errorHandler = defaultErrorHandler) => {
      const values = (x) => Object.values(x)
      const { doc, onSnapshot } = database()
      return onSnapshot(
        doc(`${path}/${fileId}`),
        handleSnapshot(withAction, fileId, path, identity, true, clientId)
      )
    }
}

listenToFlatArrayAtPath('flatCrads/${fileId}/cards')(...)
```

6 Watching for Invalid Version Changes

Plottr Pro should include a hook that triggers whenever a file's version changes. That hook should save and then refresh the window to cause the file to go through the migrator and version check in a safe state (i.e. without the file loaded.).

Note: migrating a loaded file might result in partial writes to Firebase and ought to be avoided.

7 Backwards Compatibility

This change is not backwards compatible. Making it backwards compatible would render the change vastly more complex and increase support's exposure from the feature.

8 Production Considerations

What have we learned from prior releases of Plottr? For a start, when we intended to support backwards and forwards compatibility, we've not done enough work to prove that old/new versions of Plottr DTRT (do the right thing).

In previous releases, Plottr ran into a number of problems relating to compatibility. Initially, there was no check for compatibility because it was assumed that all new versions of the PLTR file would be backwards compatible, i.e. that older versions of the program would be forwards-compatible with newer files. This assertion is flawed and very difficult to guarantee.

It's a sound and well-understood problem to make a program handle older versions of it's managed data because it can encode full knowledge of the data and all formats it once took. It's unsound to build logic that caters for all future versions because it's not possible to know what changes will be made to a file schema in the future. We decided that the best Plottr can do is check whether there is a migration ahead of the current version of Plottr that significantly changes the file format and is therefore marked as "breaking".

Another lesson we learned is that we must test candidate releases with old Plottr files and specifically check the breaking migration logic with each release. In a few instances, the migration logic was broken prior to a release. The ensuing weeks resulted in reports that files were broken because of inconsistent character attributes and other similar problems.

8.1 Plottr Mobile

Plottr Mobile changes at a much slower rate than Plottr Desktop and Plottr Web and is expected to have feature parity with Web and Desktop. We haven't established whether mobile's migration and backwards compatibility logic is working as expected. Prior to the release of this feature we need to investigate whether it's working.

If Mobile's migration and breaking change logic is broken, we ought to fix and release mobile prior to releasing this feature.

9 Conclusion

In order to support arbitrarily large timelines, Plottr needs to migrate files stored on Firebase. After migration, a file's cards will exist in a flat array. The cards will be segmented into sub collections using the id of the file, which will also be used to restrict access to the sub collection as per Plottr's usual permission rules. In addition to the new rule, we'll add a breaking migration to `pltr` and support for listening to an array from a path in `plottr_firebase`. Finally, whenever Plottr boots a pro file, it will need to migrate any cards it finds in the old document to a new document. Special care should be taken to ensure that our migration logic is functional on all platforms before releasing to production.